Currency Vision / DhanDrishti

Currency Vision is an accessibility-focused currency recognition system. It identifies banknote currency and denomination from an uploaded image or camera frame, reports confidence, converts detected values into another currency, stores account history, and can provide localized voice feedback.

The repository contains two related interfaces:

- `Currency_Vision`: the model demo and Node/Express inference API.
- `Frontend`: the full React/Vite application with authentication, onboarding, camera scanning, results, conversion, history, preferences, voice features, and counterfeit-checking screens.

The trained model recognizes 36 banknote classes across USD, PHP, INR, EUR, AUD, and CAD.

For presentation material, see PITCH.md.

Features

- Camera-based banknote scanning in the React application.
- Image upload and browser-side ONNX inference in the standalone demo.
- Server-side ONNX inference through ONNX Runtime Node for the React scanner.
- Currency and denomination detection with confidence filtering.
- Single-currency, multiple-currency, and automatic detection modes.
- Currency conversion using exchange-rate data, with application-level fallback behavior.
- Account signup, login, profile lookup, and saved preferences.
- Account-specific detection history stored in MongoDB.
- Localized result and onboarding text through the frontend i18n layer.
- Browser speech recognition for supported voice navigation flows.
- Optional ElevenLabs text-to-speech and speech-to-text through a server-side proxy.
- Counterfeit/security-check views in the React application.

Repository Layout

```
.
├── README.md
├── QUICKRUN.md
├── Currency_Vision/
│   ├── index.html                # Standalone browser demo
│   ├── package.json              # Vite, API, and model dependencies
│   ├── server.ts                 # Express API and ONNX Runtime Node inference
│   ├── public/
│   │   ├── model.onnx            # Trained YOLO ONNX model
│   │   ├── ort/                  # ONNX Runtime browser assets
│   │   └── samples/              # Optional sample images
│   ├── src/
│   │   ├── banknotes.ts          # Model class map and conversion helper
│   │   ├── inference.ts          # Browser ONNX inference path
│   │   ├── main.ts               # Standalone demo entry point
```

```
│   │   └── style.css              # Standalone demo styling
│   ├── dataset.yaml               # 36-class YOLO dataset definition
│   ├── ARCHITECTURE.md             # System diagrams and design details
│   ├── requirements.txt            # Python training dependencies
│   └── Yolov8s-SE.yaml             # YOLO model/training configuration
└── Frontend/
    ├── package.json               # React/Vite scripts
    ├── vite.config.ts             # Port 3000 and /api proxy configuration
    └── src/
        ├── pages/                 # Application routes and screens
        ├── components/            # Shared UI and feature components
        ├── services/              # Inference, conversion, voice, and TTS
services
        ├── context/               # Application and voice state providers
        └── constants/              # Supported currencies and app constants
```

Architecture

The production-style local flow is:

```
Browser camera
    -> Frontend React scanner
    -> POST /api/inference
    -> Express + Sharp preprocessing
    -> ONNX Runtime Node + public/model.onnx
    -> Detection result
    -> React result/converter/history/voice UI
```

The API also exposes authentication, profile, history, and TTS proxy routes. MongoDB stores users and history. ElevenLabs is accessed only by the server when TTS is configured; the API key must never be placed in frontend code.

The standalone `Currency_Vision` demo uses a separate browser inference path:

```
Uploaded image
    -> ONNX Runtime WebAssembly
    -> public/model.onnx
    -> Detection boxes and currency totals
    -> Optional conversion display
```

See Currency_Vision/ARCHITECTURE.md for the Mermaid diagrams, request sequence, techniques, and proposed-system comparison.

Requirements

- Windows, macOS, or Linux.
- Node.js with npm. Use a current LTS release.

- MongoDB running locally or a reachable MongoDB deployment for account and history features.
- A modern browser with camera permission support for scanning.
- Optional: an ElevenLabs API key for voice synthesis/transcription.
- Optional for model training: Python and the packages in `Currency_Vision/requirements.txt`.

The API uses these defaults when environment variables are not supplied:

- API port: `8787`
- MongoDB URI: `mongodb://127.0.0.1:27017`
- MongoDB database: `currency_vision`

Installation

Install each JavaScript project independently:

```
cd Currency_Vision
npm install

cd ..\Frontend
npm install
```

The repository root does not define an application script. Commands must be run from the project directory that owns the relevant `package.json`.

Running the Applications

Full React application

Start the API in one terminal:

```
cd Currency_Vision
npm run api
```

Start the React frontend in a second terminal:

```
cd Frontend
npm run dev
```

Open the URL printed by Vite, normally `https://localhost:3000` because the frontend enables the basic SSL plugin. The Vite server proxies `/api` requests to `http://localhost:8787`.

For a detailed copy-paste checklist, see QUICKRUN.md.

Standalone browser demo

The standalone demo runs browser-side inference and does not require the Express API or MongoDB:

```
cd Currency_Vision
npm install
npm run dev
```

Open the Vite URL, choose an image, and review the detected banknotes. This path uses ONNX Runtime WebAssembly and `public/model.onnx`.

Production-style builds

Build the API project and standalone demo:

```
cd Currency_Vision
npm run build
npm run preview
```

Build the React frontend:

```
cd Frontend
npm run build
npm run preview
```

The React build is written to `Frontend/out`. The `Currency_Vision` build uses the default Vite output directory unless the project configuration changes it.

Environment Configuration

Create or update `Currency_Vision/.env` for the API:

```
PORT=8787
MONGODB_URI=mongodb://127.0.0.1:27017
MONGODB_DB=currency_vision
ELEVENLABS_API_KEY=replace-with-your-server-side-key
```

Do not commit `.env` files or expose provider keys in `Frontend` source code. If a real API key has been exposed, revoke it and create a replacement before deploying.

For a separately hosted model API, configure the React frontend with a Vite environment variable before starting it:

```
VITE_MODEL_API_URL=http://localhost:8787
```

When developing locally with the default Vite proxy, leave `VITE_MODEL_API_URL` empty and use relative `/api` requests.

API Reference

Health

```
GET /health
```

Returns a small readiness response such as `{ "ok": true, "model": "currency-yolo" }`.

Inference

```
POST /api/inference
Content-Type: image/jpeg | image/png | application/octet-stream
```

The request body is the raw image. Optional headers are:

- `X-Detection-Currency`: preferred single currency.
- `X-Detection-Currencies`: comma-separated allowed currencies.
- `X-Detection-Mode`: `single`, `multiple`, or `automatic`.
- `X-Confidence-Threshold`: numeric result threshold.

The API resizes input to `640 x 640`, removes alpha, converts pixels to normalized RGB tensor planes, runs the ONNX model, filters detections, and returns status, selected detections, total value, and possible low-confidence reasons.

Authentication and profile

```
POST /api/auth/signup
POST /api/auth/login
GET  /api/auth/profile?email=user@example.com
```

Signup requires name, email, phone, and a password of at least six characters. Passwords are salted and hashed with Node.js `scrypt` before storage.

History

```
GET    /api/history?email=user@example.com
POST   /api/history
DELETE /api/history/:id?email=user@example.com
DELETE /api/history?email=user@example.com
```

History records are scoped by normalized email and are upserted by record ID.

Text-to-speech and transcription

```
POST /api/tts
```

Supported multipart/form actions include `voices`, `transcribe` with an audio file, and `synthesize` with a voice ID, text, language code, and optional speed. These operations require `ELEVENLABS_API_KEY`.

Model and Dataset

`Currency_Vision/dataset.yaml` defines 36 classes:

- USD: 1, 5, 10, 20, 50, 100
- PHP: 20, 50, 100, 200, 500, 1000, and a new 1000 class
- INR: 10, 20, 50, 100, 200, 500, 2000
- EUR: 5, 10, 20, 50, 100, 200
- AUD: 5, 10, 20, 50, 100
- CAD: 5, 10, 20, 50, 100

The class order in `Currency_Vision/src/banknotes.ts` must remain aligned with the trained model and `dataset.yaml`. Reordering labels without retraining or updating the model will produce incorrect denominations.

The committed `public/model.onnx` is used at runtime. Training notebooks and YOLO configuration are included for experimentation; the dataset itself is expected to be supplied separately according to `dataset.yaml`.

Scripts

`Currency_Vision`

Command	Purpose
`npm run dev`	Start the standalone Vite demo.
`npm run api`	Start the Express model/API server on port 8787.
`npm run build`	Run TypeScript compilation and build the standalone frontend.
`npm run preview`	Preview the built standalone frontend.

`Frontend`

Command	Purpose
`npm run dev`	Start the React/Vite app on port 3000 with `/api` proxying to port 8787.
`npm run dev:phone`	Alias for the frontend development server.

Command	Purpose
`npm run build`	Build the React application into `out`.
`npm run preview`	Preview the React production build.
`npm run lint`	Run ESLint with warnings treated as errors.
`npm run type-check`	Run the application TypeScript check without emitting files.

Troubleshooting

The scanner cannot reach the model API

1. Confirm the API terminal is running from `Currency_Vision`.
2. Open `http://localhost:8787/health` and confirm it returns JSON.
3. Confirm the React app is running on port `3000` and that its Vite proxy targets port `8787`.
4. If using a remote API, set `VITE_MODEL_API_URL` and restart Vite.

MongoDB errors during signup or history loading

Start MongoDB locally or set `MONGODB_URI` to a reachable deployment. The model inference route can still be tested independently, but account and history routes need MongoDB.

Voice features report a provider error

Set a valid server-side `ELEVENLABS_API_KEY` in `Currency_Vision/.env`, restart `npm run api`, and verify that the key has access to the requested ElevenLabs operations.

Camera access is blocked

Allow camera permission for the frontend origin. Use the HTTPS URL printed by the Vite development server when required by the browser, and make sure no other application is holding the camera.

Detection is empty or low confidence

Use a clear, well-lit image with the note facing the camera. Avoid folded notes, sharp angles, heavy glare, and partial occlusion. The server-side route applies a model confidence filter and can return `low_confidence` with suggested causes.

Development Notes

- The frontend uses the `@` alias for `Frontend/src`.
- API calls use relative `/api` paths by default so the Vite proxy works locally.
- The server accepts image uploads up to 12 MB for inference and TTS audio uploads.
- MongoDB collections are `users` and `history` in the configured database.
- Exchange-rate requests are made by the currency helper and may fail when the network is unavailable.
- The standalone browser demo and the React application use different inference paths; changes to one path do not automatically change the other.

Security and Privacy

- Keep `ELEVENLABS_API_KEY` server-side and outside source control.
- Use HTTPS and appropriate authentication/session hardening before public deployment.
- The current API uses email values supplied by the client to scope profile and history requests; add real server-side sessions or signed tokens before treating it as production authentication.
- Do not use detected currency results as the sole basis for high-risk financial decisions. Lighting, note condition, camera quality, and model coverage affect accuracy.

License and Data

No license or dataset distribution terms are defined in the repository. Confirm the rights to the banknote images, trained weights, third-party services, fonts, and other assets before redistribution or commercial deployment.